Regre_Lineal040726

July 4, 2026

[2]:
```
# Importar librerias

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import linear_model
from sklearn.metrics import mean_squared_error, r2_score
```

[3]:
```
# Cargar mis datos
data = pd.read_csv("articulos_ml.csv")
```

[4]:
```
# Ver la cantidad de renglones y columnas
data.shape
```

[4]:
```
(161, 8)
```

[5]:
```
# Ver los primeros renglones de mi dataset
data.head()
```

[5]:
```
                                               Title  \
0  What is Machine Learning and how do we use it …
1   10 Companies Using Machine Learning in Cool Ways
2  How Artificial Intelligence Is Revolutionizing…
3  Dbrain and the Blockchain of Artificial Intell…
4  Nasa finds entire solar system filled with eig…

                                                 url  Word count  # of Links  \
0  https://blog.signals.network/what-is-machine-l…        1888           1
1                                                NaN        1742           9
2                                                NaN         962           6
3                                                NaN        1221           3
4                                                NaN        2039           1

   # of comments  # Images video  Elapsed days  # Shares
0            2.0               2            34    200000
1            NaN               9             5     25000
2            0.0               1            10     42000
3            NaN               2            68    200000
```

```
4          104.0                4            131     200000
```

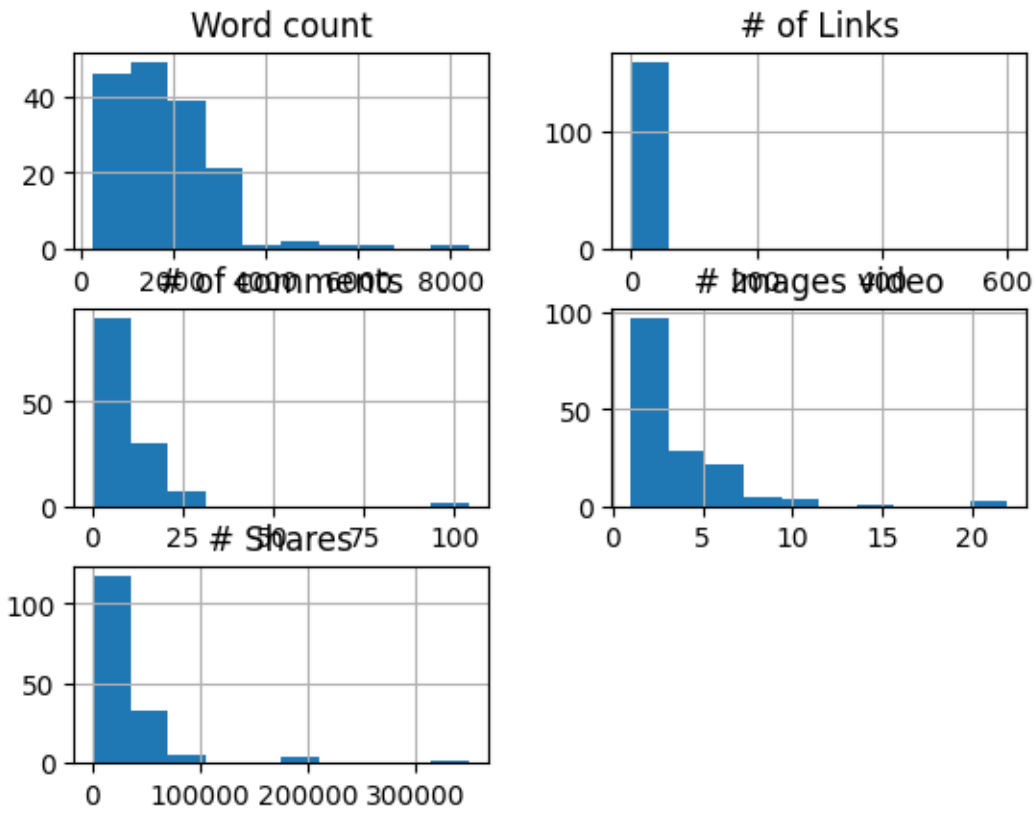
[6]:
```
# Estadistica descriptiva de las variables númericas
data.describe()
```

[6]:
```
        Word count  # of Links  # of comments  # Images video  Elapsed days  \
count   161.000000  161.000000     129.000000      161.000000    161.000000
mean   1808.260870    9.739130       8.782946        3.670807     98.124224
std    1141.919385   47.271625      13.142822        3.418290    114.337535
min     250.000000    0.000000       0.000000        1.000000      1.000000
25%     990.000000    3.000000       2.000000        1.000000     31.000000
50%    1674.000000    5.000000       6.000000        3.000000     62.000000
75%    2369.000000    7.000000      12.000000        5.000000    124.000000
max    8401.000000  600.000000     104.000000       22.000000   1002.000000

            # Shares
count     161.000000
mean    27948.347826
std     43408.006839
min         0.000000
25%      2800.000000
50%     16458.000000
75%     35691.000000
max    350000.000000
```

[9]:
```
# vamos a hacer histogramas de cada variable excepto: Title, url y elapsed days

data.drop(['Title','url','Elapsed days'], axis = 1).hist()
plt.show()
```



[10]:
```
#Estamos filtrando para quedarnos solo con los renglones que cumplan ambas␣
 ↪caracteristicas_
# Menor o igual a 3500 en Word count y menor o igual a 80000 en # Shares
filtered_data = data[(data['Word count'] <= 3500) & (data['# Shares'] <= 80000)]
```

[11]:
```
filtered_data.shape
```

[11]:
```
(148, 8)
```

[12]:
```
colores = ['orange', 'blue']
```

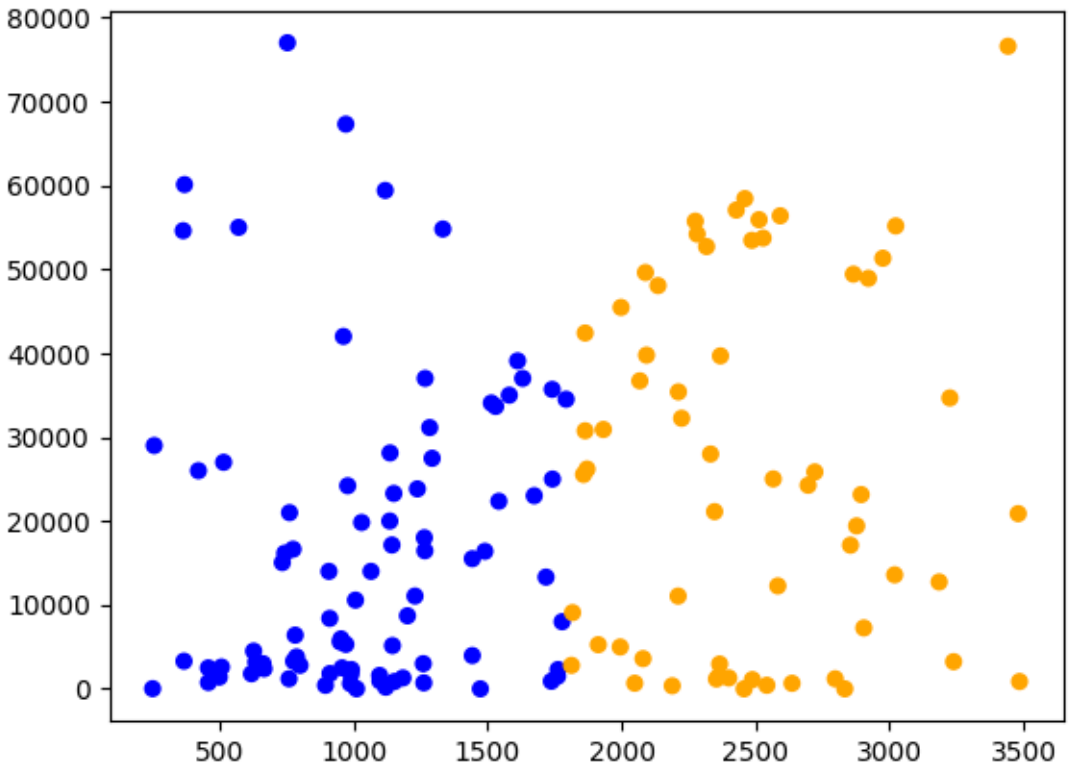
[14]:
```
f1 = filtered_data['Word count'].values
f2 = filtered_data['# Shares'].values
```

[17]:
```
asignar = []

for index, row in filtered_data.iterrows():
    if(row['Word count'] > 1808):
        asignar.append(colores[0])
    else:
        asignar.append(colores[1])
```

```
plt.scatter(f1,f2, c=asignar, s=30)
```

[17]: <matplotlib.collections.PathCollection at 0x13c550e20>



[18]:
```
# Crear mis datos de entrada para el modelo
dataX = filtered_data[['Word count']]
X_train = np.array(dataX)
```

[23]:
```
dataX
```

[23]:

	Word count
1	1742
2	962
5	761
7	753
8	1118
..	…
156	3239
157	2566
158	2089
159	1530

```
160         953

[148 rows x 1 columns]
```

[24]:
```
y_train = filtered_data['# Shares'].values
```

[25]:
```
# Crear mi variable que va a contener lo necesario para realizar el modelo
regr = linear_model.LinearRegression()
```

[26]:
```
# Entrenar el modelo con la función fit
regr.fit(X_train, y_train)
```

[26]:
```
LinearRegression()
```

[27]:
```
# Revisar la pendiente calculada
print('Pendiente: ', regr.coef_)
```

```
Pendiente:  [5.69765366]
```

[28]:
```
# Revisar la intersección de la linea
print('Intersección de la linea: ', regr.intercept_)
```

```
Intersección de la linea:  11200.30322307416
```

[29]:
```
# Generar predicciones
y_pred = regr.predict(X_train)
```

[37]:
```
len(y_pred)
```

[37]:
```
148
```

[30]:
```
y_train[:5] # valores reales, respuestas correctas
```

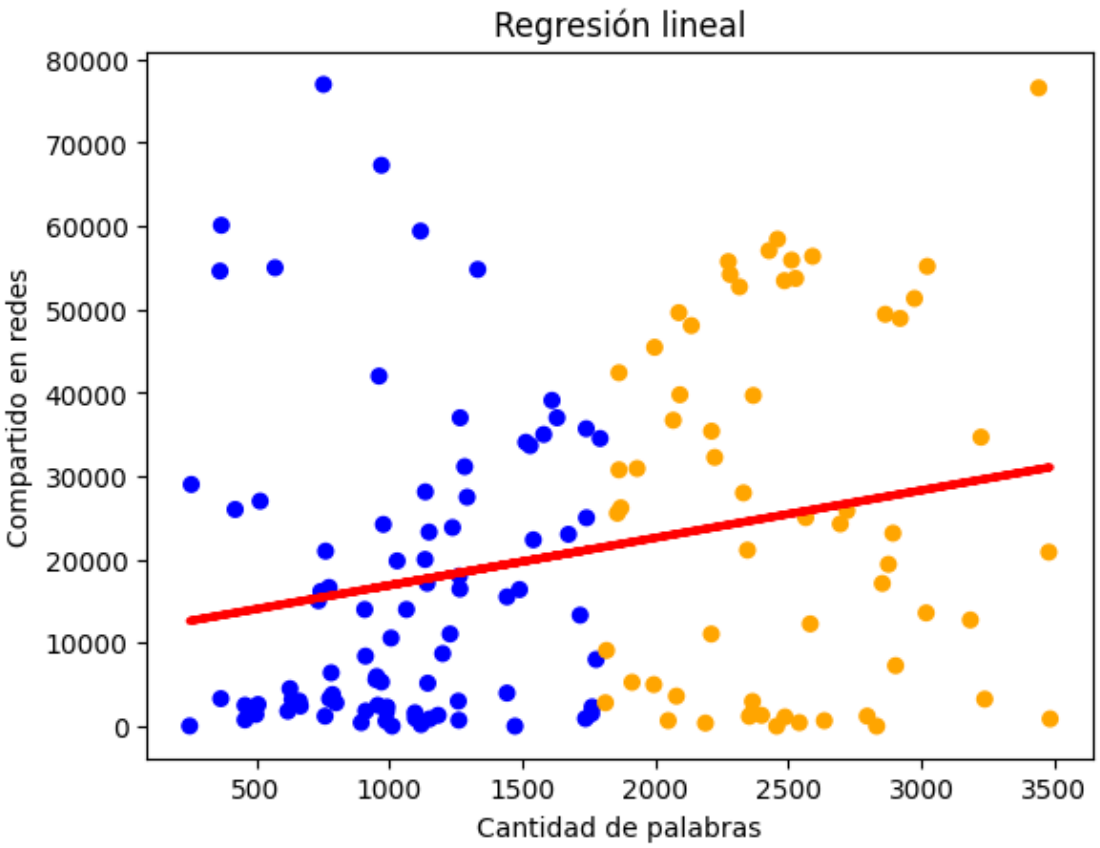
[30]:
```
array([25000, 42000, 21000, 77000, 59400])
```

[31]:
```
y_pred[:5] # valores de la predicción
```

[31]:
```
array([21125.61589425, 16681.44604148, 15536.21765635, 15490.63642709,
       17570.28001204])
```

[34]:
```
# Graficar la linea
plt.scatter(X_train[:,0], y_train, c=asignar, s=30)
plt.plot(X_train[:,0], y_pred, color='red', linewidth=3)
plt.xlabel('Cantidad de palabras')
plt.ylabel('Compartido en redes')
plt.title('Regresión lineal')
```

[34]:
```
Text(0.5, 1.0, 'Regresión lineal')
```



[35]:
```
y_pred_2mil = regr.predict([[2000]])
y_pred_2mil
```

[35]:
```
array([22595.61053785])
```

[36]:
```
# Parte de la evaluación
r2_score(y_train, y_pred)
```

[36]:
```
0.05519842281951404
```

[ ]:
```

```